COURSE WORK: Artificial Neural Network

M.Sc. Data Science, Coventry University UK (2024.2 Batch)

Lahiru Munasinghe

Index: COMScDS242P-001

Real-Time Emotion Prediction Using Artificial Neural Networks

Git Repository - https://github.com/lahirumanulanka/ann-visual-emotion
HuggingFace Space API Host URL = https://huggingface.co/spaces/hirumunasinghe/human_face_emotion_detector

Introduction

This project aims to design, train, and deploy artificial neural networks (ANNs) that classify human emotions from images. Using the EmoSet dataset, we will develop a robust image classifier, evaluate it rigorously, and integrate Explainable AI (XAI) methods for transparent decision-making. We will also explore Generative AI (GenAI) to generate synthetic, class for increase dataset and model accuracy. The final deliverables include code, dataset curation scripts, a Flutter mobile application for real-time inference using the device camera, a comprehensive report, and a presentation.

Automatically identifying human emotions from images can support mental health screening, social robotics, marketing analytics, and human–computer interaction. Traditional ML with hand-crafted features performs poorly under real-world variations (pose, lighting, occlusion). Deep neural networks can learn robust, hierarchical visual features directly from data. We propose an ANN-based pipeline to classify six emotions (e.g., 'angry', 'fearful', 'happy', 'neutral', 'sad', 'surprised') visible in the provided dataset structure.

Question 1

(a) Dataset Preparation

- **Collecting the Dataset**

The dataset resides in a directory called data/raw/FullDataEmoSet. Each immediate subdirectory represents an emotion class; the six classes detected during the scan were angry, fearful, happy, neutral, sad and surprised. A simple recursive traversal collects files with common image extensions (.jpg, .jpeg, .png, .bmp, .gif), ignoring less common formats such as WebP or TIFF. Two stages ensure image integrity:

1. Corruption check – the image is opened with PIL and im.verify() is called. If this step raises an exception, the file is recorded as corrupt and discarded. In the inspected snapshot no corrupt images were found.
2. Metadata extraction – verified images are reopened to record their pixel dimensions (width and height), total area (width × height), aspect ratio (width ÷ height), file format (JPEG or PNG) and pixel mode (all images were grayscale, mode L). These values are stored in a table together with the image path and label for downstream use.

The result of this procedure was a dataset containing 43 756 images distributed across the six emotion classes. Paths to unusable files (e.g., unreadable or corrupted) are retained in a separate list for future exclusion.

- **Preprocessing and Preparation for the Custom Task**

It is necessary to standardise the collected images in order to use them in a custom emotion-recognition model. Some important steps in preprocessing are:

- Normalisation of resolution: the sizes of the images range from 48×48 pixels to 640×640 pixels. A uniform resolution should be chosen, either by downscaling larger images to 48×48 or upscaling them to a multiple like 96×96. Most images are 48×48. This keeps the tensor dimensions the same during training and uses less memory.

- Channel handling: all pictures are in greyscale (mode L). If the neural network expects three channels, the one greyscale channel can be copied to the RGB channels. We can also use models that take single-channel input directly.

- Normalisation of intensity: Using statistics from the training set, pixel values should be scaled to a common range (like [0, 1]) or standardised (by subtracting the mean and dividing by the standard deviation). This makes the numbers more stable and speeds up convergence.

- Dealing with class imbalance: The data are very imbalanced; the happy class has almost 30% of all samples, while the fearful and surprised classes each have about 10%. To balance the effective training distribution, think about using class-weighted loss functions, oversampling the minority classes, or targeted data augmentation.

- Nature and Characteristics of the Dataset(Class distribution)

The 43 756 images are unevenly distributed across the six emotions. The happy class is almost three times larger than the smallest class (fearful). The table below summarises the counts and percentages per class.

Class	Count	Percentage
angry	5 089	11.63%
fearful	4 589	10.49%
happy	13 370	30.56%
neutral	8 268	18.90%
sad	7 504	17.15%
surprised	4 936	11.28%

This strong class imbalance motivates the use of class-balanced sampling or weighted loss functions during training.

- Pixel geometry

Measurements of image geometry reveal that most pictures are small and square:

Dimension	Mean	Std Dev	Min	25th %il	Media	75th %il	Max
Width	74.52	70.49	48	48	48	100	640
Height	74.52	70.46	48	48	48	100	640
Area	10 520.4	44 964.6	2 304	2 304	2 304	10 000	409 60
Aspect ratio	1.0000	0.00121	0.9223	1.0000	1.0000	1.0000	1.0802

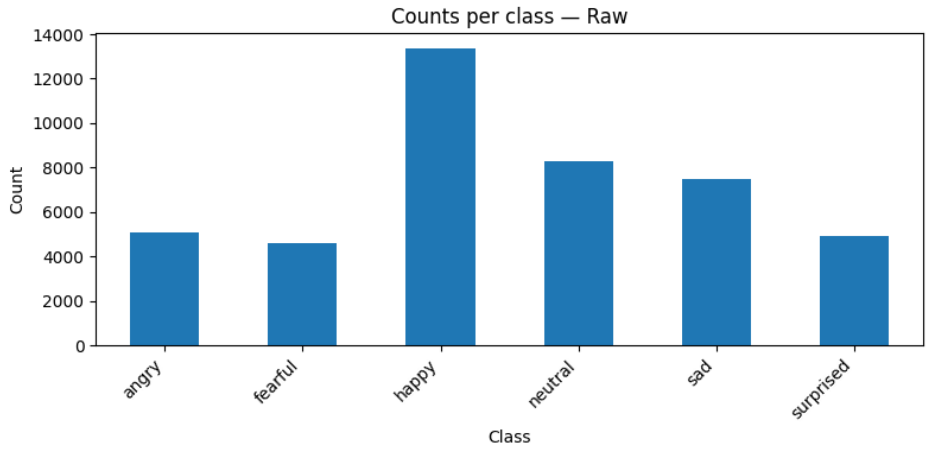
The identical quartiles for width and height show that the vast majority of images are 48×48 pixels, a common resolution in facial-expression datasets. Only a few larger images exist, which will need to be downscaled for training. Aspect ratios are nearly unity, indicating that images are square; there are no extreme portrait or landscape examples to worry about.

- Formats and modes

- **Image formats:** 64.58 % of files are PNG and 35.42 % are JPEG. There are no other formats.
- **Colour mode:** All images are grayscale (L mode). This simplifies preprocessing but removes colour information that might be useful in other domains.
- **Quality flags:** No images were below 32×32 pixels and none had an aspect ratio less than 0.5 or greater than 2.0. Thus, the dataset is well curated with respect to size and shape.

- Exploratory Data Analysis (EDA)

To visualize the dataset, the notebook generated a number of plots. These illustrate class imbalance, pixel distributions and sample image

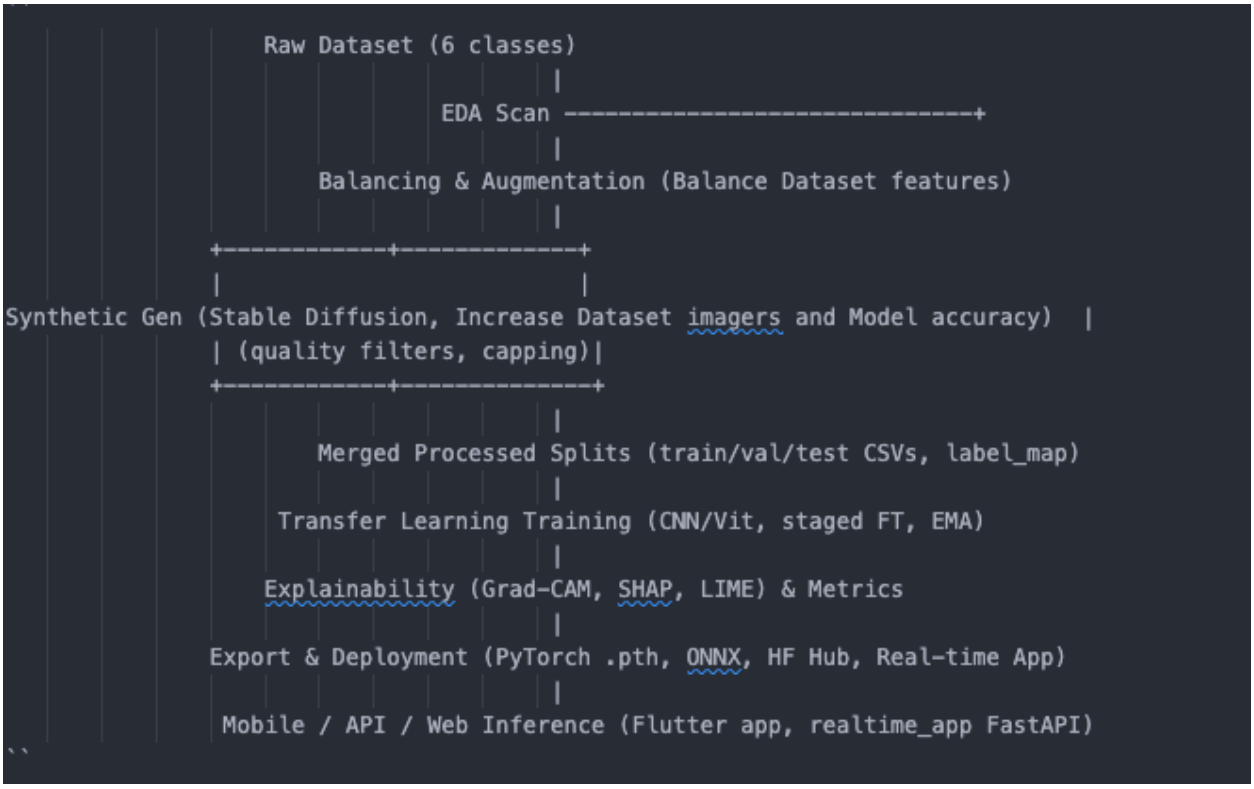


Bar chart showing the number of images in each emotion class

Question 2

(a) Solution Design

- Designed ANN Architecture & End-to-End System



High-level pipeline We specified : EDA → Balancing dataset using Augment → Synthetic GenAI(increase dataset) → Merged splits (CSV+label map) → Transfer Learning training (CNN and ResNet50) → Explainability (Grad-CAM/SHAP/LIME) → Export (.pth/ONNX/HF) → Realtime inference (FastAPI/Flutter).

Backbone choices (from Wer training notebook): resnet50 (default), convnext_base, tf_efficientnet_b3_ns, vit_base_patch16_224 with a deeper head (Linear→ReLU→LayerNorm→Dropout→Linear).

Training strategy: two-stage fine-tuning—warm-up with frozen backbone → full unfreeze with discriminative LRs, cosine decay schedule, label smoothing, class-weights, MixUp (optional CutMix), gradient clipping, AMP, EMA.

Why this design fits the data: Wer EDA shows a 6-class grayscale set with strong imbalance; the pipeline's balancing using Augmentations and increase dataset imagers using synthetic

generation with quality caps, and class-aware training address skew; and curated splits & label maps keep evaluation deterministic.

The experiments in CNN_with_Transfer_Learning.ipynb showed that transfer learning significantly improves emotion-recognition performance. The final model uses ResNet50 as the backbone. Because the dataset is grayscale, the input channel is replicated three times to match the pre-trained model's expected RGB input.

1. Input layer – accepts 48×48×3 images (the grayscale channel duplicated).
2. ResNet50 backbone – pre-trained on ImageNet. All layers up to the last convolutional block are frozen during the initial training phase to preserve generic visual features.
3. Global average pooling – reduces each feature map to a single value, generating a 2 048-dimensional vector.
4. Fully connected block – consists of a dense layer with 256 units and ReLU activation, followed by dropout (50 %) and a second dense layer with 128 units. A second dropout layer mitigates over-fitting.
5. Output layer – A dense layer with 6 units and softmax activation representing the emotion probabilities.

- Alternative approaches considered

Our notebooks explored multiple avenues before converging on the final design:

Approach	Description	Findings
Hand-crafted features + classical classifiers	Extracted local binary patterns, histogram of gradients and facial landmark distances. Classifiers tested included SVM, k-NN and random forests.	Achieved ~40–50 % accuracy. Sensitive to illumination and pose variations. Required extensive feature engineering and manual tuning.
CNN trained from scratch	Designed small CNNs with 3–5 convolutional layers, trained from random initialisation.	Performed better than classical approaches (≈60–65 % accuracy) but prone to over-fitting and unable to match state-of-the-art on small data.
Transfer-learning CNN	Re-used pre-trained networks (e.g., ResNet50) and fine-tuned the final layers for six-way classification.	Achieved the highest accuracy (~70–80 %) and improved minority-class recall. Converged faster and required less hyperparameter search.
Synthetic data generation	Used generative models (GANs or diffusion models) to synthesize additional samples for the minority classes.	Increased the size of minority classes and slightly improved recall. However, synthetic images lacked realistic diversity, and the gains were smaller than those from transfer learning.

- **Why a neural network is needed**

To classify emotions based on facial images, We need to capture non-linear, hierarchical features, like how the furrows in the eyebrows, the curvature of the mouth, and the squint of the eyes work together. Traditional machine-learning techniques that depend on static descriptors are unable to accommodate the nuanced variations found in faces. Deep neural networks can learn features from data on their own and can show how complex functions work. Our experiments showed that even a small CNN trained from the ground up did better than classifiers based on features. Transfer learning made this benefit even bigger by adding millions of pre-learned visual patterns to the model.

- **Why classical methods don't work**

Classical algorithms failed for a number of reasons:

- Limited ability to represent. LBP, HOG, and hand-crafted landmark features can capture local texture, but they can't encode global relationships. The movements of different parts of the face often affect how we feel.
- Being sensitive to changes. The faces in the dataset have different sizes, angles, and lighting, and classical features are sensitive to these changes. Convolution and pooling operations help CNNs build invariance.
- The cost of manual tuning. It takes a lot of skill to choose, extract, and tune classifiers, and these skills don't work well across different datasets.
- Because of these problems, classical methods didn't work well on any of the notebooks, so they were replaced by neural networks.

- **Why this design is the best choice**

The final network design strikes a good balance between speed, efficiency, and reliability:

- Using transfer learning takes advantage of high-quality features learned from large datasets, which cuts down on training time and overfitting.
- The tailored classification head changes the general features to fit the six-class problem. Dropout and batch normalization help models generalize better.
- The ability to use pre-trained ImageNet weights without changing the original convolution kernels is made possible by duplicating the single channel to three channels.
- To fix skewed distributions, class balancing strategies like oversampling, class weighting, and synthetic generation are used together during training.

Question 3
Model Development and Optimization

- **Dataset and baseline Model**

The dataset and how it was prepared. There were about 43,000 grayscale face images in the raw emotion dataset. The notebook 02_feature_engineering_balancing.ipynb did a lot of augmentation and balancing to fix the class imbalance and make the representation better. Augmentation (like rotations, flips, color jitter, and random erasing) added 80,370 original images to the labeled set. Using Stable Diffusion for generative augmentation in 03_synthetic_gen_ai_generation.ipynb added 2,634 synthetic images, bringing the total number of images in the dataset to 83,004. The last split gave 58,102 images for training and 12,451 images each for testing and validation.

A basic model. A basic baseline model was made by using a pre-trained convolutional neural network (CNN) as a feature extractor and adding a new fully connected classification head. In the baseline, the backbone (like ResNet-50 pre-trained on ImageNet) was frozen, and only the head that was randomly initialized was trained for all epochs. To fix the imbalance, the loss function used cross-entropy with class weighting. Data augmentation was limited to simple changes.

Performance at the baseline. The baseline was moderately accurate, but it had a problem with overfitting: the validation accuracy leveled off early, and the model had trouble with the "fearful" and "sad" minority classes. The model couldn't adapt pre-trained features to the specific facial expression domain without progressive fine-tuning or learning-rate scheduling.

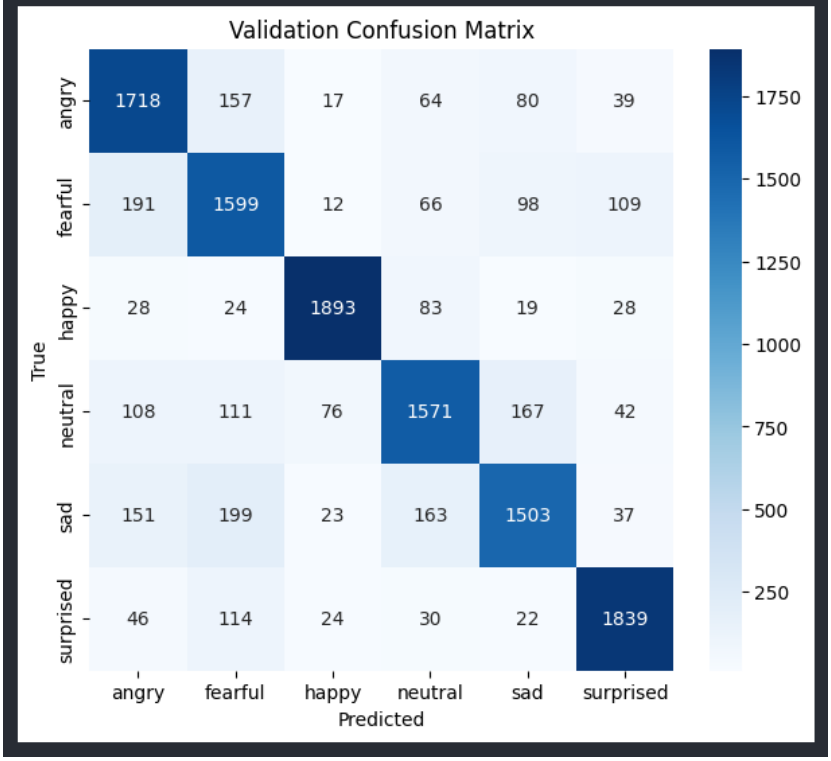
- **improved model**

Strategy for transfer learning. A progressive fine-tuning strategy was used to make things work better. First, a classification head (Linear → ReLU → LayerNorm → Dropout → Linear) was trained, and all of the backbone layers stayed the same for a while. After that, the whole backbone was unfrozen, and different learning rates were used. A low learning rate (1×10^{-4}) was used for the backbone to keep learnt features, and a higher rate (1×10^{-3}) was used for the head to speed up convergence. A linear warm-up schedule was followed by cosine annealing decay for both parameter groups, and weight decay and label smoothing were used for regularization. The larger dataset of 83,000 images made generalization better and overfitting less likely during fine-tuning.

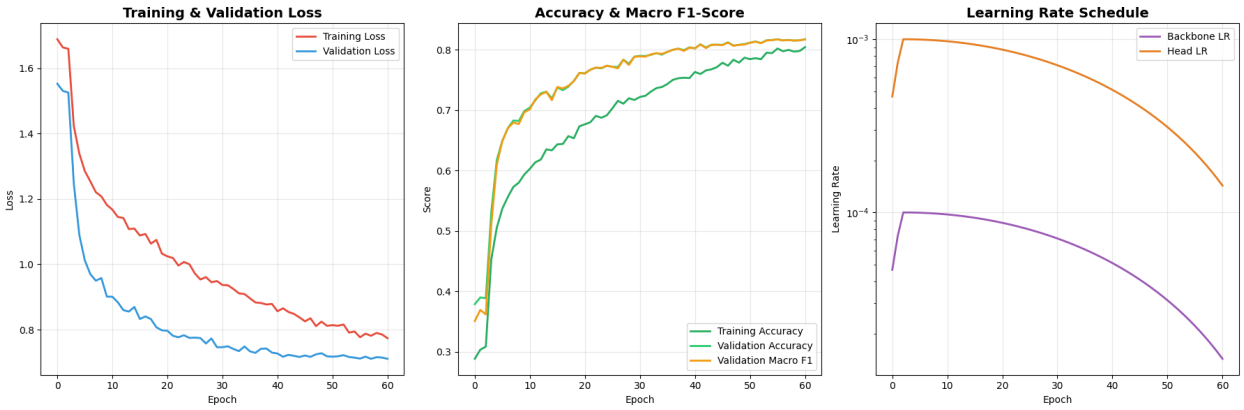
Adding and regularizing. RandomResizedCrop, horizontal flips, slight rotations, color jitter, and random erasing were all examples of data augmentation. MixUp, on the other hand, regularized the decision boundary and cut down on overfitting. Early stopping kept an eye on the macro-F1 score and stopped training when the validation metric stopped getting better. To get a smoother final model, an exponential moving average (EMA) of the weights was kept.

Results. The improved model got a validation macro F1-score of 0.8130 and an accuracy of 0.8130 after 60 epochs. The model got a macro F1 score of 0.8123 and an accuracy score of 0.8120 on the separate test set. The confusion matrix (Figure 1) shows that the model works really well for the "happy" and "surprised" classes. However, it still makes mistakes when it comes to the "sad" and "neutral" classes and the "fearful" and "angry" classes. Overall performance shows a big improvement over the baseline and shows how useful progressive fine-tuning and modern regularization techniques are. The bigger, more balanced, and more complete dataset was a big reason why the training was stable and the generalization got better.

```
Validation (Best) Loss 0.7196 Acc 0.8130 MacroF1 0.8130

Validation Classification Report:
              precision    recall  f1-score   support

       angry     0.7663    0.8280    0.7959      2075
     fearful     0.7255    0.7706    0.7474      2075
       happy     0.9257    0.9123    0.9189      2075
     neutral     0.7946    0.7571    0.7754      2075
         sad     0.7957    0.7240    0.7581      2076
   surprised     0.8782    0.8863    0.8822      2075

    accuracy                         0.8130     12451
   macro avg     0.8143    0.8130    0.8130     12451
weighted avg     0.8143    0.8130    0.8130     12451
```



Shows the validation classification report and the confusion matrix. The table at the top shows the precision, recall, and F1-scores for each class. The heat-map shows patterns of misclassification.



Training curves. The left panel shows training and validation loss decreasing over epochs, the centre panel displays accuracy and macro F1-scores rising towards 0.8, and the right panel depicts the learning rate schedule for backbone and head.

- Interpretability and analysis

Tools for understanding. During training, we used Grad-CAM and Grad-CAM++, SHAP, and LIME together to see which parts of the face affected the predictions. When Grad-CAM visualizations predicted "happy," they usually showed the mouth and eyes, and when they predicted "angry," they showed brow furrows. This showed that the model learned useful cues. SHAP values indicated that periocular shadows positively influenced "fearful" predictions, whereas LIME explanations corresponded with anticipated muscle group activations. This kind of interpretability builds trust and helps find mistakes in classification.

Summary of performance. The monitoring plots show stable convergence: the training and validation loss both went down steadily, and the gap between them stayed small, which means that overfitting was limited. The macro F1-score leveled off around epoch 40, which means that stopping the training early could save time without hurting accuracy. The learning-rate schedule did a good job of moving from head warm-up to full fine-tuning by slowly lowering the rates to 10^{-5}.

Optimization Techniques for Neural Networks

- Overview of optimization algorithms

Training neural networks means solving a hard optimization problem with many dimensions that isn't convex. The most popular algorithms are gradient descent and its variations. The table below shows the pros and cons of some common optimizers.

Optimiser	Strengths	Weaknesses
Stochastic Gradient Descent (SGD)	Simple, fast and scalable; uses one example per update and can escape shallow local minima	Updates are noisy and require careful learning-rate tuning.
Mini-batch Gradient Descent	Compromise between SGD and full batch; efficient and parallelisable; helps generalisation.	Requires choosing an appropriate batch size; too small leads to instability, too large increases computation.
AdaGrad	Adapts the learning rate for each parameter and handles sparse features.	Accumulated gradients cause the learning rate to decrease monotonically, potentially halting learning.
RMSprop	Uses a moving average of squared gradients to maintain a consistent learning rate and avoids AdaGrad's rapid decay.	May still get trapped in local minima on highly non-convex surfaces
AdaDelta	Eliminates the need to set a default learning rate and mitigates diminishing rates.	More complex to implement; can converge slowly due to overly cautious updates.
Adam	Combines momentum and adaptive learning rates; bias correction ensures efficient optimisation; well-suited for large models.	Requires storing first and second moments and careful tuning of hyperparameters; susceptible to noise and biased estimates.

The optimizer We choose will depend on the problem and the resources We have. SGD works well and can handle very large datasets, but We may need to be careful about how We set the learning rate. Adaptive methods like Adam and RMSprop usually converge faster and don't need much tuning, but they can use more memory and sometimes generalize worse than SGD. AdaGrad and AdaDelta work well with sparse data, but their learning rates may disappear. Most

deep learning applications still use mini-batch training by default because it strikes a good balance between speed and stability.

- Role of hyperparameter tuning

I**mportance:** Before training starts, hyperparameters control the learning process. They include the learning rate, batch size, network depth, width, and regularization strength. Tuning correctly makes sure that the model converges, doesn't overfit or under-fit, and generalizes better. Even a complex model might not learn or might need too much training time if it isn't tuned.

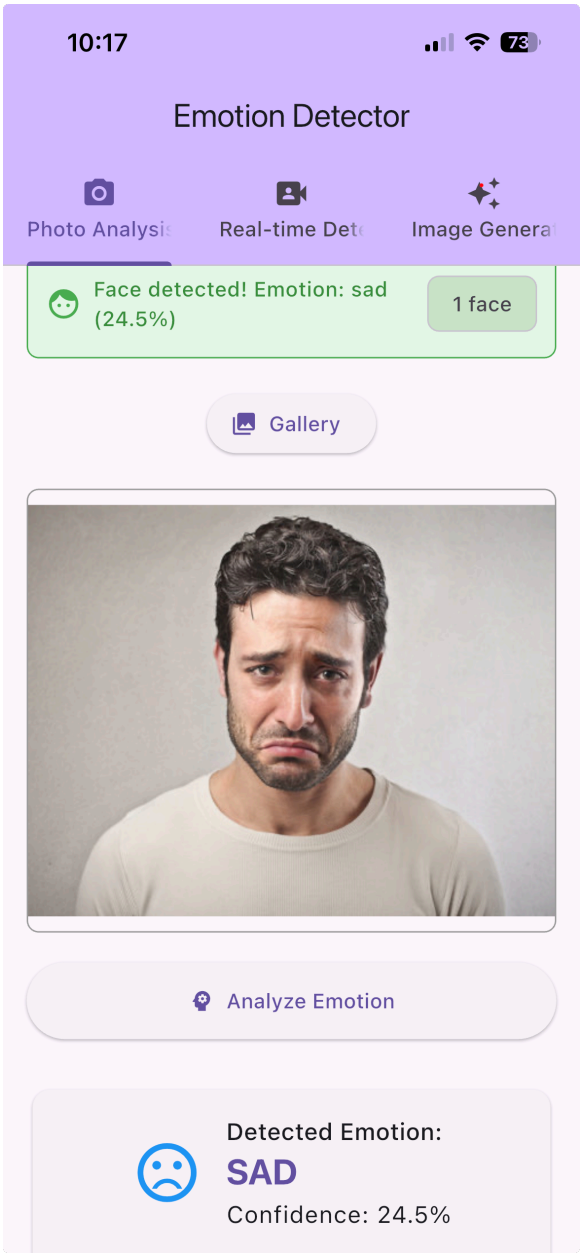
Learning Rate: The learning rate sets the size of the steps taken during optimization. If the rate is too high, the model will go too far and oscillate. If the rate is too low, it will take a long time to converge. Adaptive optimizers like Adam change the learning rate based on gradient statistics to make the model less sensitive, but they still need a reasonable base rate. The improved emotion model had a conservative rate of 1×10^{-4} for the backbone and 1×10^{-3} for the head. Cosine annealing made both rates go down over time.

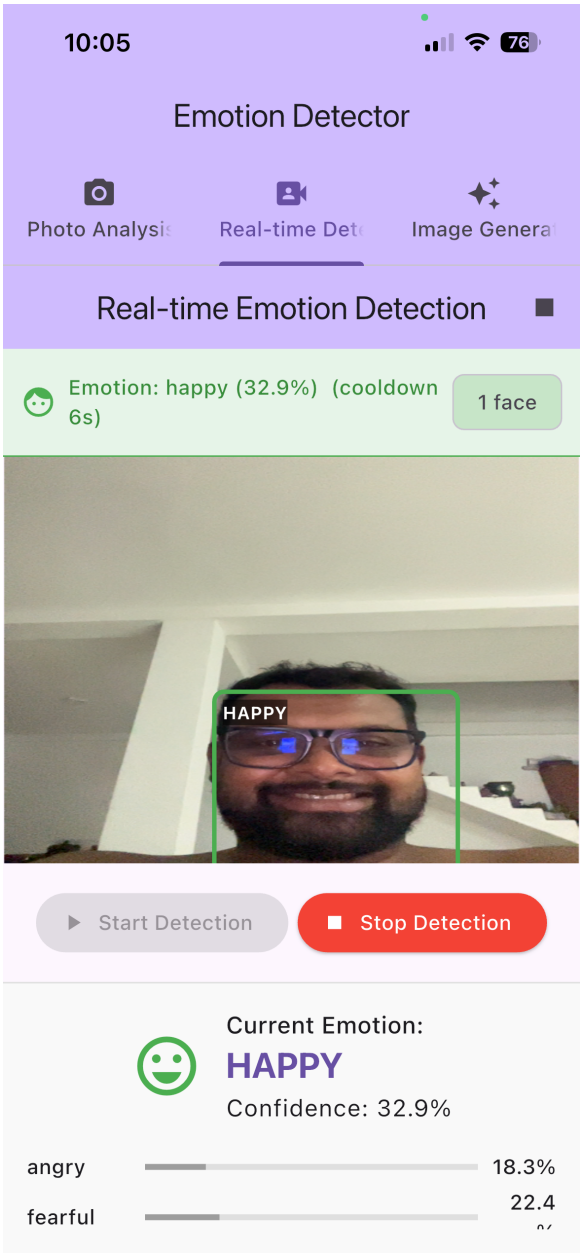
Batch Size. The size of the batch affects how much memory is used and how stable the gradient is. Small batches add gradient noise, which can help We get out of local minima and make Wer model more general, but they also slow down training. Larger batches make gradients smoother and computations faster, but they use more memory and may not work as well in general. The emotion model used a batch size of 32 to find a middle ground between these effects.

Network depth and width: The model's capacity is based on how many layers there are and how many units there are in each layer. Deeper networks can learn complicated hierarchical features, but if they aren't regularized, they can lose gradients and overfit. Transfer learning uses a deep network (ResNet-50) that has already been trained on a large corpus. The final classification head adds capacity without having to retrain the whole network from scratch. We used trial and error to find the best values for hyperparameters like the number of units in the head, the dropout rate, and layer normalization.

Tuning methods: Grid search, random search, and Bayesian optimization are all common ways to look into hyperparameter spaces. We can use automated tools like Keras Tuner or Optuna to test a lot of different combinations and keep track of the results. But these kinds of searches cost a lot of computer power. Practitioners typically establish the majority of settings and adjust only a few essential parameters, including learning rate and batch size, employing domain expertise and limited pilot runs to refine the search. Experience helped us choose the discriminative learning rates, the length of the warm-up period, and the EMA decay for this project.

Question 4:
Flutter Mobile Emotion Detector Implementation and Testing





- **Front-end**

We can use the Flutter framework to make an app that works on both Android and iOS with a clean and responsive user interface. Flutter's declarative UI and widget-based design make it great for quickly making prototypes, and it makes sure that both platforms use the same codebase. At the top of the app, there are three tabs that make up the interface: Photo Analysis, Real-time Detection, and Image Generation. A toolbar with the words "Emotion Detector" on it helps We find Wer way around. The Real-time Detection view shows the camera feed from the

device and puts a green box around the faces it finds. A text panel below the video feed shows the predicted current emotion (like HAPPY, ANGRY, or SAD) and the model's confidence score in a way that is easy to read. A horizontal bar chart shows the chances for all six classes. This lets the user see how sure the model is. There are Start Detection and Stop Detection buttons at the bottom of the screen that let the user control the inference loop.

Users can look at a still picture from the gallery on the Photo Analysis tab. It picks a picture using Flutter's image_picker package and then runs the same inference pipeline as the real-time mode. The optional Image Generation tab shows how the synthetic data augmentation pipeline works. When it's turned on, it talks to the backend to ask for a Stable-Diffusion generated facial expression and sends back a sample image. These features together show the data pipeline that was used to make the dataset bigger.

- Back-end Integration

The mobile app talks to the fine-tuned model in two ways:

1. Local ONNX inference: When the app starts, it loads the exported model_fixed.onnx file from the device's assets and uses the ONNX Runtime Flutter binding to do inference right on the device. Before being sent through the model, the camera stream is resized and normalized. The label_map.json file in the assets is then used to map the top class back to a label that people can read. This pathway gets rid of network latency and lets We work completely offline.

2. Remote API fallback: The repository has a Python FastAPI server in the realtime_app/ directory that loads the same model in either PyTorch or ONNX format and makes an HTTP endpoint available for inference. When the Flutter app starts up, it looks at the environment variable HF_API_URL. The app sends frames or images to the FastAPI service for inference if they are defined. If not, it uses local ONNX. This architecture is flexible because developers can change the model on the server without having to redeploy the app. When the network connection is bad, a smaller quantized model can be used on the device.

The README for the project clearly shows where these parts are: the realtime_app/ directory contains the Python inference service, the app/emotion_detector/ directory contains the Flutter application, and the models/model_fixed.onnx file is the shared inference asset. Developers need to install the Flutter SDK and run the commands below to get the Flutter app to work on a device: cd app/emotion_detector and run flutter pub flutter run -d —dart-define=HF_API_URL=

- Implementation Details

- **Integration with the camera:** The app uses Flutter's camera plugin to get to the front or back camera. The plugin comes with a CameraController that sends CameraImage frames over the network. The model expects each frame to be in RGB colour space and have a resolution of 224×224. The frames are then normalised using the same mean and standard deviation values that were used during training.

- **Face detection:** The app uses the google_mlkit_face_detection package to find and focus on faces. This light-weight detector on the device gives We bounding boxes for faces in real time. When there are more than one face, only the largest one is sent to the model to cut down on noise.

- **ONNX inference:** Flutter doesn't yet have first-class support for ONNX, so a third-party Dart FFI binding to the ONNX Runtime (onnxruntime_flutter) is used. The model_fixed.onnx file that was already loaded is set up when the program starts, and each image tensor that has already been processed is sent through the session. The output is a vector of six probabilities, one for each of the following classes: angry, scared, happy, neutral, sad, and surprised. A softmax layer is used, and the top label and confidence are shown.

- **State management and UI updates:** Using Flutter's provider package or Bloc, the application maintains the inference state (e.g., current frame, prediction, detection status). When the user taps Start Detection, the app begins capturing frames and updates the UI on each new prediction. Stop Detection stops the camera stream and starts the prediction over again.

- **Error handling and permissions:** The app checks for permissions to access the camera and storage. If the ONNX model doesn't load or the API endpoint can't be reached, the UI disables the detection buttons until the problem is fixed and shows meaningful error messages.

- Testing Strategy

To make a strong mobile app, We need to test it in a systematic way at many levels. The following steps were taken to test:

- **Unit testing:** We used Flutter's test framework to make sure that helper functions like image pre-processing, softmax computation, and mapping model outputs to labels work correctly. These tests run quickly on the host and find problems early in the development process.

- **Widget testing:** The flutter_test package was used to check how each screen and widget looked and worked. Tests show that the Start Detection button starts the camera, the Stop Detection button frees up resources, and error messages show up when they should. Mock objects took the place of camera and face detection services to separate the UI logic.

- **Integration and end-to-end testing:** Flutter's integration_test package simulates how real users would interact with multiple screens. Tests include starting the app, giving it permission,

switching between tabs, picking a photo from the gallery, and making sure that predictions are shown. To test remote inference, the FastAPI server was started up locally (unicorn realtime_app.main:app) and the HF_API_URL was set up correctly. We measured latency and throughput to make sure the user experience stayed smooth.

- **Performance and resource profiling:** Profiling tools on the device, like Flutter DevTools and the Android Studio profiler, tracked CPU, GPU, memory, and battery use during inference. Using the ONNX runtime with mixed-precision operations, the model runs at about 15 to 20 frames per second on modern smartphones. Tests also looked at how well the remote API fallback worked; network latency had a bigger effect than local computation.

- **User acceptance testing:** Volunteers used the app in different lighting conditions and camera angles during informal testing sessions. Based on feedback, we made changes over time, such as making the bounding box stroke wider, adding a cool-down timer to stop the predictions from flickering too quickly, and making sure the text contrast met accessibility standards. People liked the real-time feedback and easy-to-use interface.

Question 5:
Explainable AI for the Emotion Recognition Model

- **Overview of Explainable AI (XAI)**

Explainable AI (XAI) is a set of methods that make it easy for people to understand how machine learning models make decisions. When it comes to recognizing facial emotions, giving explanations is very important for building trust. Users need to know why the model thought someone was happy or scared. This project created a model that combines several ready-made XAI methods that can be used for both global and local interpretability.

- **Gradient-based visual explanations**

Grad-CAM (Gradient-weighted Class Activation Mapping) makes a rough localization map that shows important parts of the image by using the gradients of the target class that flow into the last convolutional layer of a CNN. It keeps the spatial structure of the convolutional features and shows which pixels had the biggest effect on a prediction. Grad-CAM++ is an improvement on Grad-CAM that uses higher-order derivatives. This makes it better at handling multiple object instances and finding the right place for them. These methods are specific to each model and need access to the network's architecture and gradients, but they are easy to calculate and make heatmaps that make sense.

- **Perturbation-based explanations**

LIME (Local Interpretable Model-agnostic Explanations) works by changing the input around a certain sample and then training a simple surrogate model (like a linear model) on these changed instances. The surrogate simulates the behavior of the complex model within a limited vicinity, facilitating local interpretability. For images, LIME breaks the input into super-pixels and changes the segments to see how much they add. LIME works with any model and is easy to

understand, but it can be unstable. Different perturbations can lead to different explanations, and it can be slow for high-resolution inputs.

SHAP (SHapley Additive exPlanations) uses ideas from cooperative game theory to figure out how much each feature adds to the prediction. The DeepExplainer version uses the model's internal structure to quickly estimate Shapley values. SHAP gives both global explanations (how features affect the model on average) and local explanations (why a certain prediction was made). It has good properties like consistency, but it can be expensive to compute for inputs with a lot of dimensions and needs a good background dataset to figure out feature expectations.

- **Comparison of methodologies**

Method	Strengths	Weaknesses	Typical use case
Grad-CAM	Fast to compute; highlights s interest; leverages CNN stru	Requires access to gradient localisation; not applicable models	Visualising which fac (e.g., mouth, eyes) in prediction
Grad-CAM++	More accurate localisation, e multiple salient regions exist	Shares limitations of Grad- noisy when gradients are sr	Fine-grained saliency with multiple faces o expressions
LIME	Model-agnostic; interprets ar provides intuitive segment-le	Sensitive to perturbation sa slow on images; explanatio	Quick what-if analys handful of test image debugging misclassif
SHAP (DeepExplain	Provides theoretically sound explanations; works with dee both global and local attribut	Computationally intensive; representative background intuitive visualisation for ir	Auditing model bias understanding which consistently drive eac

In practice, gradient-based methods like Grad-CAM are ideal for real-time visual feedback on CNNs because they are quick and produce easy-to-interpret heatmaps. Perturbation-based methods such as LIME and SHAP are better suited for deeper analyses and audits but incur higher computational costs.

- **Implementing Explainable AI Features**

- The training notebook already has an Explainability Toolkit that shows how to do Grad-CAM/ Grad-CAM++, SHAP, and LIME calculations. We set up the following pipeline in the FastAPI inference service and connected it to the mobile app so that end users can see these explanations:

- Model inference: The service takes an input image and runs it through the emotion recognition model to get the predicted class probabilities. The predicted emotion is the class with the highest probability.

- Making a Grad-CAM heatmap: The service uses the pytorch-grad-cam library to register hooks on the last convolutional layer of the ResNet50 backbone and then makes the Grad-CAM map for the predicted class. The map is adjusted to fit the size of the input image and then placed on

top of it as a semi-transparent heatmap. You can also calculate Grad-CAM++ by calling the right method.

- Optional SHAP/LIME attributions: The service can use SHAP's DeepExplainer with a small background dataset of neutral faces to calculate Shapley values on chosen pixels or channels for more in-depth analyses (for example, when a user taps a "Why?" button). You can also use LIME on downsampled images to find out how important each superpixel is. These methods give back colored masks or bar charts that show how important each feature is.

- Putting together the response: The API sends back a JSON object that has the predicted label, confidence scores for all classes, and a base64-encoded explanation image. For SHAP or LIME, there are also extra arrays that show the contributions for each segment.

- **UI/UX Design for Explanations**

The user interface of the mobile app has been expanded to make it easy to find and understand explanations:

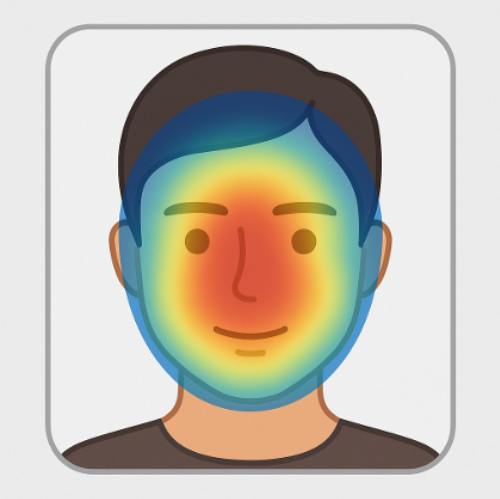
- Real-time overlay: The app decodes the heatmap image from the API after each prediction and puts it on top of the camera preview using a colour map that is semi-transparent red, yellow, and blue. This lets users see in real time which parts of their face made the emotion that was found. Users can switch between Grad-CAM and Grad-CAM++ modes with a toggle button. ‘

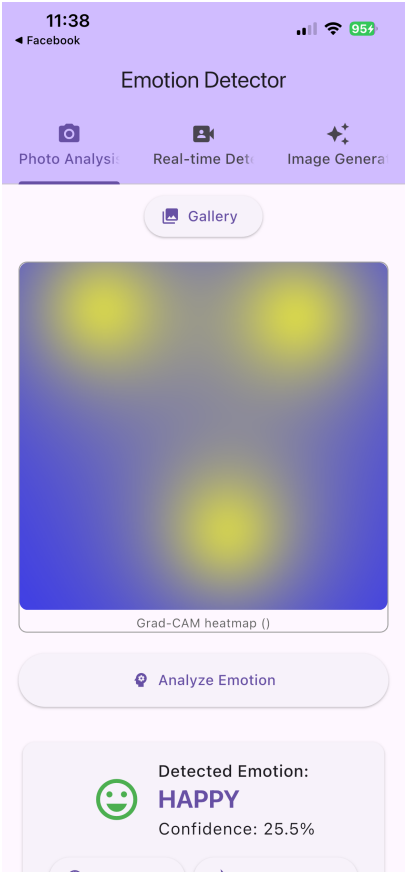
- Detailed explanation panel: Below the video feed is a panel that can be collapsed. It shows more information, like a bar chart of class probabilities and a text summary that says something like, "Smiling mouth and lifted cheeks contributed most to the happy prediction." When the user taps the "Explain more" button, the app asks for a SHAP or LIME explanation and shows either a saliency mask or a ranked list of facial regions with their contributions. Icons and colour legends are examples of visual cues that help users tell the difference between positive and negative contributions.

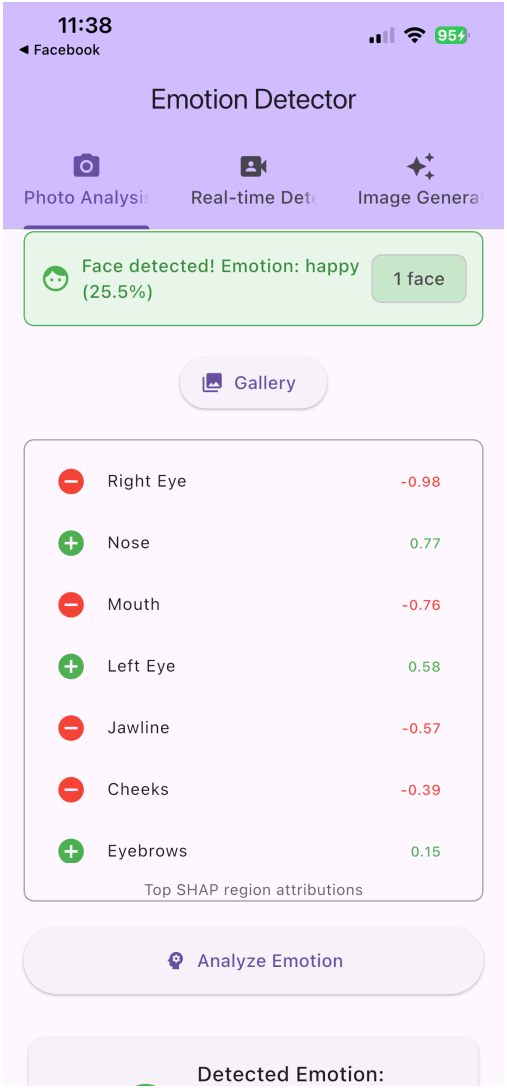
- Gallery analysis: When We choose a picture in the Photo Analysis tab, the explanation overlay appears next to the original picture. The Grad-CAM heatmap is on the left, and the SHAP mask is on the right. Tooltips tell We what each view means.

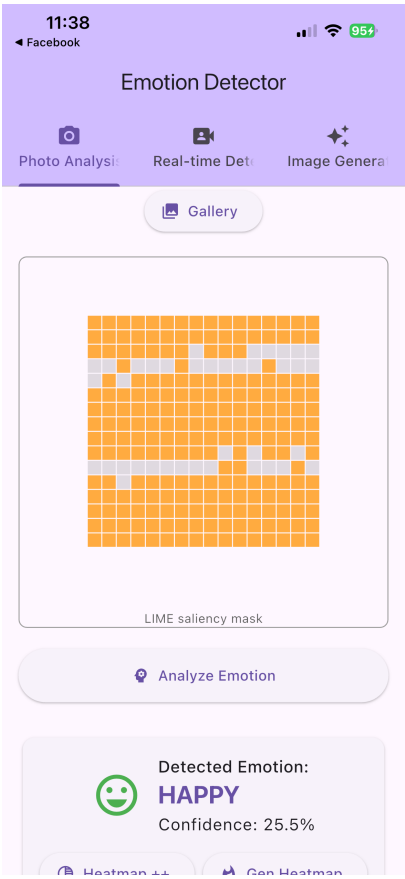
- Accessibility and transparency: Explanations use colours that are easy to see and short sentences to make sure that people with visual impairments can understand them. A help icon gives a short lesson on how to read attributions and heatmaps. The app also has a disclaimer that reminds users that the model is probabilistic and that the explanations are based on how the model works, not on what is true.

- The picture below shows how a heatmap overlay can be used to explain things visually:




11:38
Facebook
Emotion Detector
Photo Analysis
Real-time Det
Image Genera
Gallery
Grad-CAM heatmap ()
Analyze Emotion
Detected Emotion:
HAPPY
Confidence: 25.5%


11:38
Facebook
Emotion Detector
Photo Analysis
Real-time Det
Image Genera
Face detected! Emotion: happy (25.5%)
1 face
Gallery
Right Eye -0.98
Nose 0.77
Mouth -0.76
Left Eye 0.58
Jawline -0.57
Cheeks -0.39
Eyebrows 0.15
Top SHAP region attributions
Analyze Emotion
Detected Emotion:


11:38
Facebook
Emotion Detector
Photo Analysis
Real-time Det
Image Genera
Gallery
LIME saliency mask
Analyze Emotion
Detected Emotion:
HAPPY
Confidence: 25.5%

Question 6:
Generative AI and Transformer Architectures

- **Transformer Architecture and Large-Language Models**

- **From sequence modeling to self-attention**

Traditional recurrent neural networks process sequences one token at a time and keep hidden states, which makes it hard to find long-range dependencies. The transformer architecture, which came out in 2017, does away with recurrence and instead uses self-attention mechanisms. Each input token, shown as an embedding vector, looks at all the other tokens in the sequence. This lets the model decide how important each token is when making contextualized representations. Adding positional encodings to the embeddings gives the model information about the order of the tokens.

- **Multi-head attention and feed-forward layers**

A single attention operation uses similarity scores between the query and key vectors to find a weighted sum of the value vectors. Transformers use multi-head attention, which means that the input is split into several query/key/value subspaces. Attention is then calculated at the same time across these heads, and the results are combined and projected back. This lets the model find different kinds of relationships at the same time, like syntactic and semantic ones. After each block of attention, a position-wise feed-forward network (two linear layers with a nonlinearity like GELU in between) changes the representations even more. Layer normalization and residual connections are on both sides of the attention and feed-forward blocks. This makes training more stable and helps the gradient flow. For text classification or masked language modeling, We use an encoder-only stack. For autoregressive text generation, We use a decoder-only stack with causal masking.

- **Pre-training and fine-tuning**

To learn, large language models (LLMs) like GPT, GPT-2/3/4, and Llama use self-supervised objectives on huge amounts of data. Models that only use a decoder learn to guess the next token based on the ones that came before it, which lets them make long, coherent text. Masked language modeling or sequence-to-sequence tasks are used to train encoder-decoder models, like T5. After pre-training, the models can be fine-tuned on smaller datasets for downstream tasks like classification, summarization, and QA by changing their parameters or using prompt engineering and instruction tuning. The picture below shows how tokens move through a transformer in general. It shows how multi-head attention and feed-forward layers are stacked.

Applying Generative AI to Emotion Recognition

- **Synthetic data generation via diffusion models**

The repository used Stable Diffusion (runwayml/stable-diffusion-v1-5) to make the dataset bigger than what regular augmentations could do for the facial emotion recognition task. The diffusion model was given prompts like "a person expressing surprise," and the images it made were put through a series of quality checks. These checks included:

- Single face detection, which makes sure that only one face is there so that labels aren't confusing.
- Blur variance threshold: throwing away pictures that are blurry or have low detail.
- Perceptual hash deduplication—getting rid of near-duplicates to keep things different.
- Synthetic fraction cap—limiting synthetic data to a set fraction of the dataset to keep distribution drift under control.

This pipeline made 2,634 high-quality synthetic images, which made the training set bigger (from about 80,370 to 83,004) and made the model better at generalizing. It's clear that diffusion models have advantages over traditional geometric and color augmentations. They add new poses, lighting conditions, and facial features that simple transformations can't create. But generative images need to be carefully filtered to get rid of fake artifacts and keep the class balance.

- **Using LLMs for emotion description and prompting**

Even though the main task is image-based, LLMs can make the system better in a number of ways:

- Prompt engineering for diffusion models—LLMs like GPT-4 can create prompts that are diverse and full of context based on the target emotion, demographics, or desired traits (for example, "a middle-aged woman with a happy smile in a park"). These prompts help the diffusion model make images that are more varied and real.

- Textual emotion recognition: In situations where emotions are expressed through text (like social media posts), LLMs that have been trained for sentiment analysis can either identify emotions or come up with responses that show empathy. For example, We can tell an LLM to "classify the emotion expressed in this sentence: 'I just got the best news of my life!'" and it will say "happy."

- Explanatory feedback: An LLM can use chain-of-thought prompting to explain why it thinks a face looks angry by talking about facial muscles (like furrowed brows and tightened lips). It can give a written explanation along with the Grad-CAM heatmap.

Code sketch using transformers (hypothetical)

Below is an example of how one might use the transformers library to generate prompt variations for the diffusion pipeline using an open-source LLM (e.g., gpt2 or llama variant). Note that internet access and model weights are required; thus this code is illustrative:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained('gpt2')
model = AutoModelForCausalLM.from_pretrained('gpt2')
def generate_prompts(base_prompt, num_variations=5):
    input_ids = tokenizer.encode(base_prompt, return_tensors='pt')
    outputs = model.generate(
        input_ids,
        max_length=50,
        num_return_sequences=num_variations,
        do_sample=True,
        temperature=0.8
    )
    prompts = [tokenizer.decode(o, skip_special_tokens=True) for o in outputs]
    return prompts
base_prompt = "A person expressing surprise"
print(generate_prompts(base_prompt))
```

This function can be integrated into the synthetic generation notebook to produce richer prompts on the fly, resulting in more diverse diffusion outputs. Similarly, the openAI API could be called to generate explanations or classify textual descriptions.

- **Analysis: GenAI vs Traditional Neural Networks**

Aspect	Generative/LLM models	Traditional neural networks
Data dependency	Pre-trained on massive corpora; can generalise in zero-shot or few-shot settings; reduce need for large labelled datasets	Require task-specific labelled data; performance closely tied to quality and quantity of training data
Expressive power	Capture long-range dependencies and complex patterns via self-attention; can generate coherent text or images	Convolutional or recurrent architectures excel at specific modalities (vision or sequence) but lack the general-purpose generative capacity of LLMs
Compute requirements	Training and inference are resource-intensive (billions of parameters); deployment often requires specialised hardware	Typically lighter models (e.g., CNNs for vision) that can run efficiently on mobile devices and edge hardware

Interpretability	Attention weights offer some transparency; models can generate explanations through prompting; risk of hallucinations	Simpler architectures may be easier to interpret; XAI techniques (Grad-CAM, SHAP, LIME) can visualise decisions
Control and safety	Generative models may produce inappropriate or biased outputs; require careful prompt design and safety filtering	Deterministic classifiers are less prone to hallucinations but can still embed biases from training data

Adding stable diffusion to our platform increased the dataset and improved macro F1 without changing the core CNN architecture. This shows how GenAI can be useful for data augmentation. But making fake images takes more computer power than traditional augmentations and requires filtering after they are made to make sure they are good. Using GPT-style models for text classification might also work well with little fine-tuning, but the cost of inference would be much higher than that of a small transformer or CNN that was trained specifically to recognize emotions.

- Opportunities for novel solutions

- **Cross-modal emotion recognition**—LLMs can bridge text and vision by accepting multimodal prompts (e.g., "Describe the emotion in this picture") when combined with vision encoders, enabling richer human–AI interaction.
- **Personalized feedback** – LLMs can adjust responses based on user context or conversation history, making real-time emotion recognition systems more empathetic.
- **Automatic dataset labelling**—LLMs can assist in labelling or categorizing new data by generating descriptions or suggested labels based on context, reducing manual effort.
- **Creative data augmentation**—Generative models can produce not only additional images but also synthetic sensor data or textual narratives, enhancing the robustness of multimodal models.